Lunar Crater Detection Pipeline & YOLO26 Analysis

Part 1: Codebase System Architecture

The system is a highly modular, end-to-end Computer Vision framework specifically designed to process massive high-resolution lunar imagery (e.g., OHRC data), automatically annotate craters, and iteratively train YOLO26 object detection models.

Four Distinct Layers:
1. Tiling & Signal Correction (Hardware-accelerated image processing)
2. Auto-Annotation & Data Capping (Inference-based dataset generation)
3. Training & Validation (Model fitting & dataset structuring)
4. Evaluation & Benchmarking (Performance analytics & visual validation)

Part 2: Complete Code & Script Dictionary

This section explicitly breaks down the exact roles of all Python scripts identified within the repository pipeline:

--- TILING & PREPROCESSING (Layer 1) ---
* ti.py / ti1.py: Base scripts for reading raw images, calculating metadata (get_image_shape), and applying basic signal corrections to remove vertical striping artifacts.
* ti2.py: The optimized, hardware-accelerated chunk-tiling processor. It uses GPU routines (process_chunk_gpu) to normalize and stripe-correct gigabyte-scale images without exceeding VRAM, falling back to CPU when necessary.
* p1.py: Generates and structures the initial 'TileDataset' constructs for subsequent steps.

--- AUTO-ANNOTATION (Layer 2) ---
* annotate_all.py: The primary pipeline for auto-labeling. It loads the best.pt YOLO model, processes the blank tiles, converts absolute coordinates into YOLO format (to_yolo()), and generates the .txt label files.
* p2_1.py: A specialized annotation enforcement script featuring a strict dir_size_bytes() constraint to instantly halt dataset generation once a hard size limit (e.g., 8GB) is reached.
* p2_2.py: A visualization and verification script (draw_boxes_on_image) used to spot-check the bounding boxes produced by the auto-annotation layer.

--- TRAINING ORCHESTRATION (Layer 3) ---
* tra_2.py: The core training engine. It handles get_paired_files() to ensure images and labels match, executes link_or_copy() to build the final dataset folder, generates dataset.yaml, and executes the heavy YOLO training loop, ending with clear_disk_cache().

--- BENCHMARKING & PREDICTION (Layer 4) ---
* tra_1.py: Model benchmarking. Tests the resulting weights for inference speed (ms/img, FPS) and VRAM usage constraints.
* predict_preview.py: Evaluates models on random tile samples by generating a dense matplotlib preview grid comparing ground truths against predicted bounding boxes.
* predict_video.py: The final production inference script. Deploys the trained YOLO26 model onto video streams (e.g., YouTube lunar footage), saving the annotated video output frame-by-frame.

Part 3: Tiling & Annotation Execution Flows

TILING FLOW (ti2.py):
The images are too large for standard arrays. 'process_and_stream()' establishes a stream.

'compute_correction_params()' maps median pixel intensity to neutralize sensor stripes. The stream sends subsets to 'process_chunk_gpu()', outputting mathematically clean 512x512 PNGs via 'save_tile()'.

ANNOTATION FLOW (p2_1.py & annotate_all.py):
The clean PNGs are passed to the YOLO inference engine. 'load_model()' spins up the detector. For every detection, 'to_yolo()' scales the (x, y) coordinates. Concurrently, the process size is audited; if the folder exceeds the quota, dataset generation safely exits.

Part 4: YOLO26 Model Training Analysis

Hyperparameter Configuration (args.yaml):
- Model: YOLO26 Nano (yolo26n.pt)
- Training Duration: 150 Epochs
- Batch Size: 32 at 512x512 image resolution
- Optimizer: MuSGD with lr0=0.01
- Augmentations: mixup=0.15, copy_paste=0.3

Final Evaluation Metrics (Epoch 150):
- Precision (P): 75.38%
- Recall (R): 76.00%
- mAP@0.5: 84.62% (Excellent performance on standard IoU)
- mAP@0.5:0.95: 67.29% (Strong bounding box tightness)

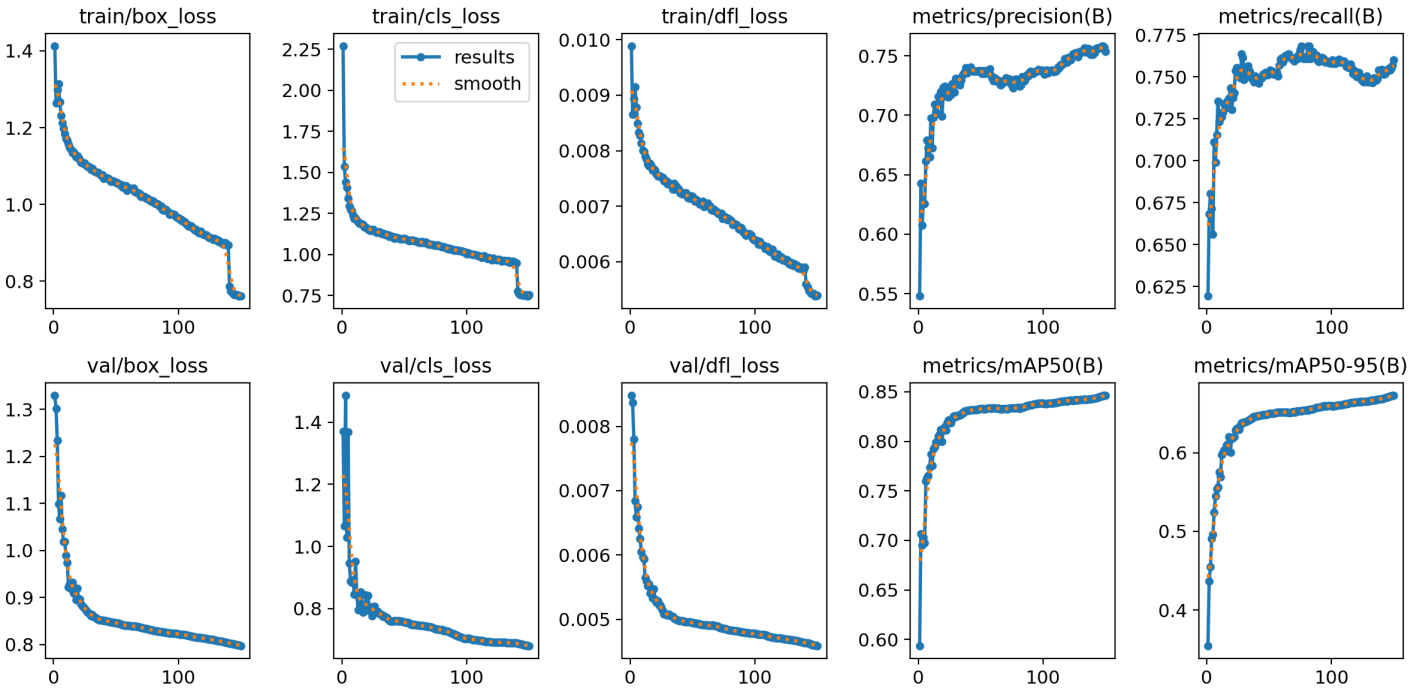
Lunar Crater Detection Pipeline & YOLO26 Analysis

Part 5: Visual Diagnostics & Predictions

Below are the visual outputs generated from the yolo26n_craters_v1-5 training directory. Each chart and batch image has been formatted to accurately display the YOLO26 validation results.

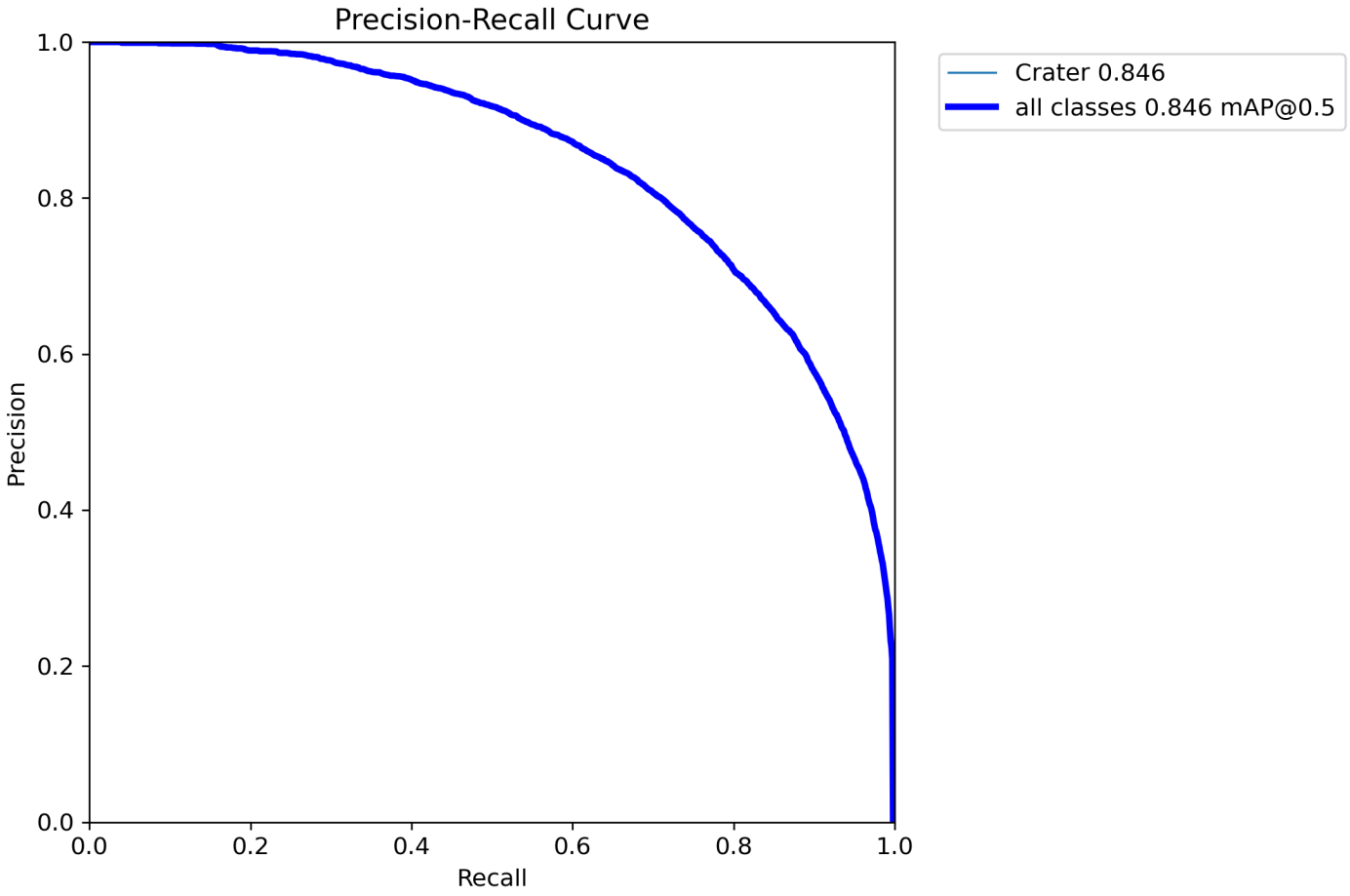
Lunar Crater Detection Pipeline & YOLO26 Analysis

Results Summary Dashboard



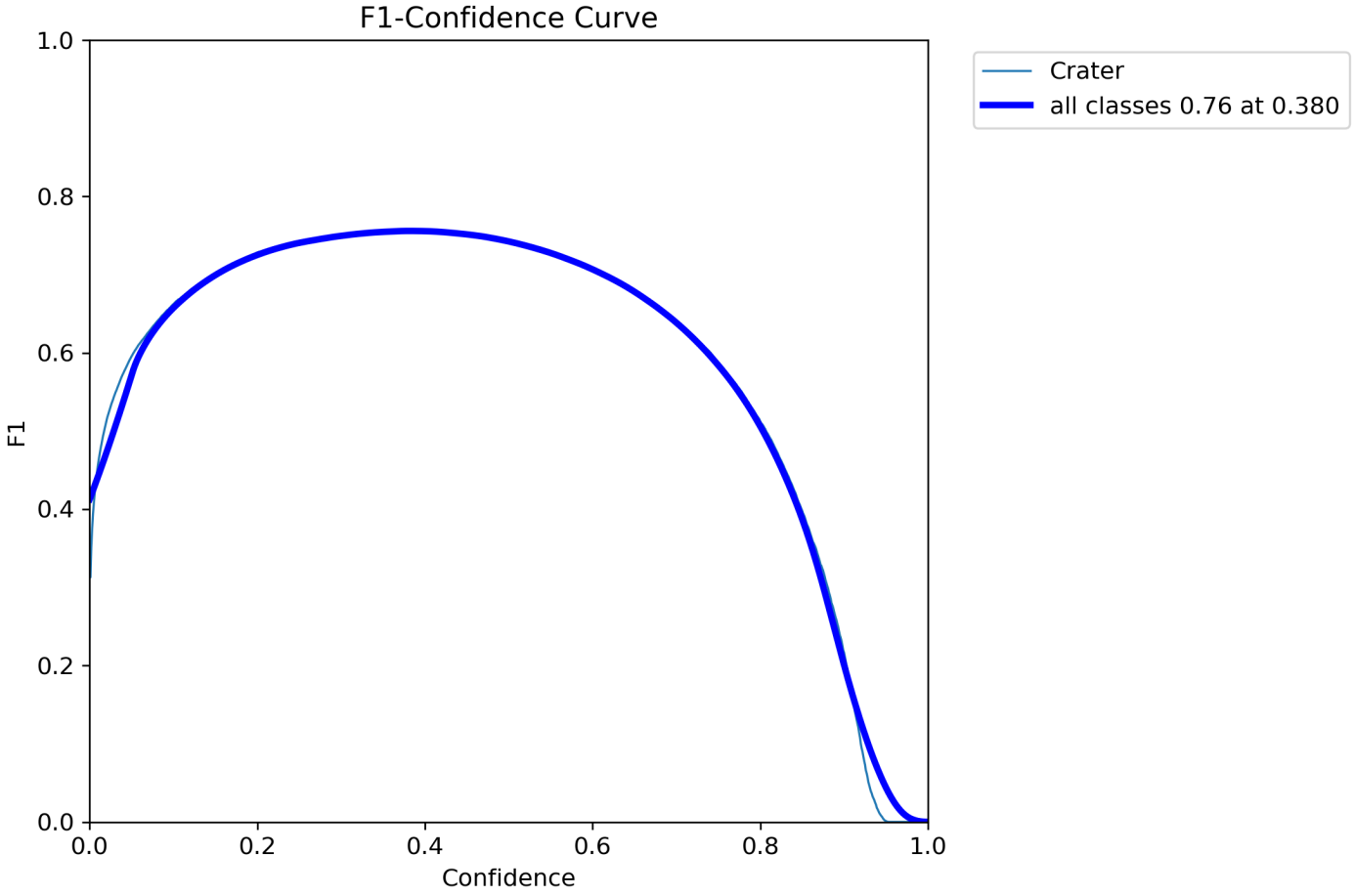
Lunar Crater Detection Pipeline & YOLO26 Analysis

Precision-Recall Curve



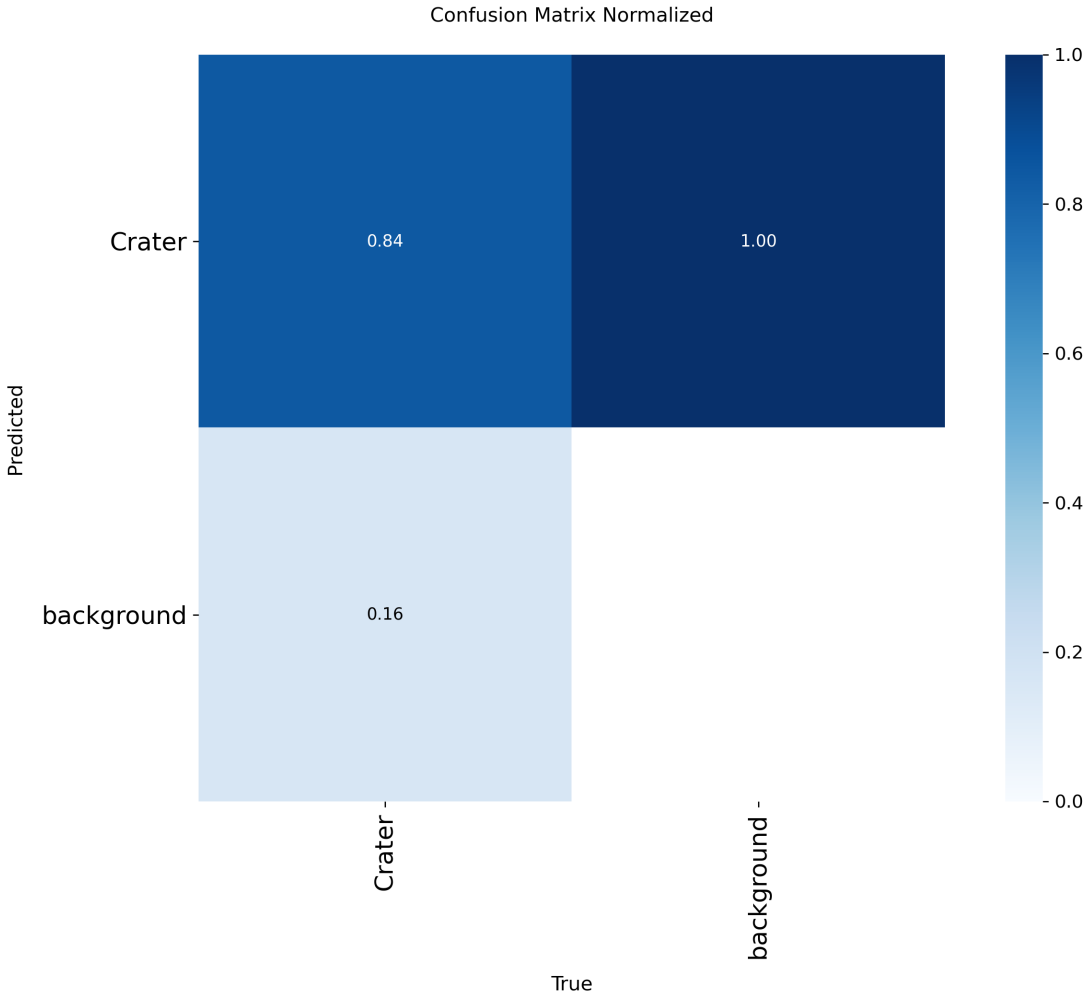
Lunar Crater Detection Pipeline & YOLO26 Analysis

F1 Score Curve



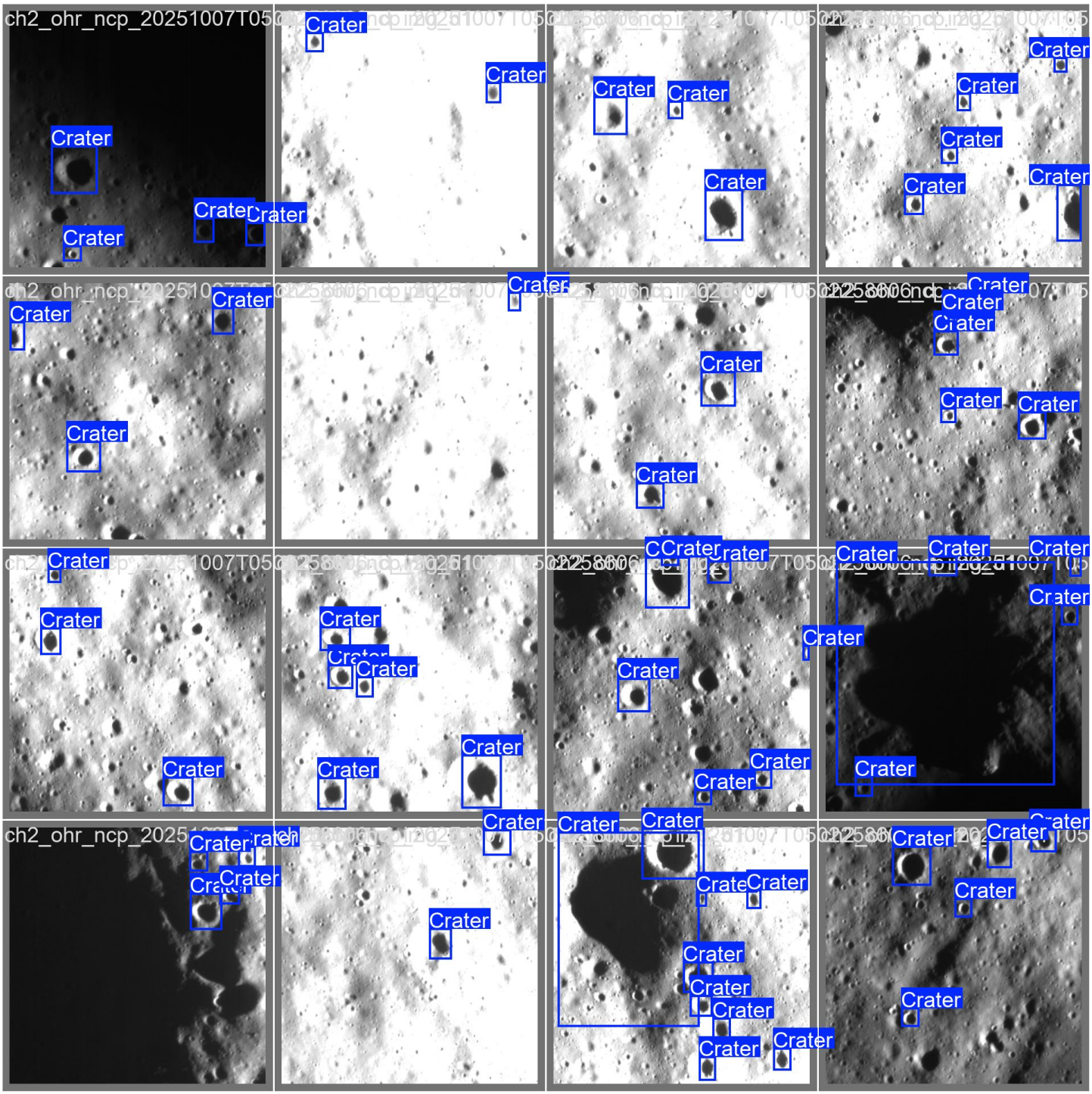
Lunar Crater Detection Pipeline & YOLO26 Analysis

Confusion Matrix (Normalized)



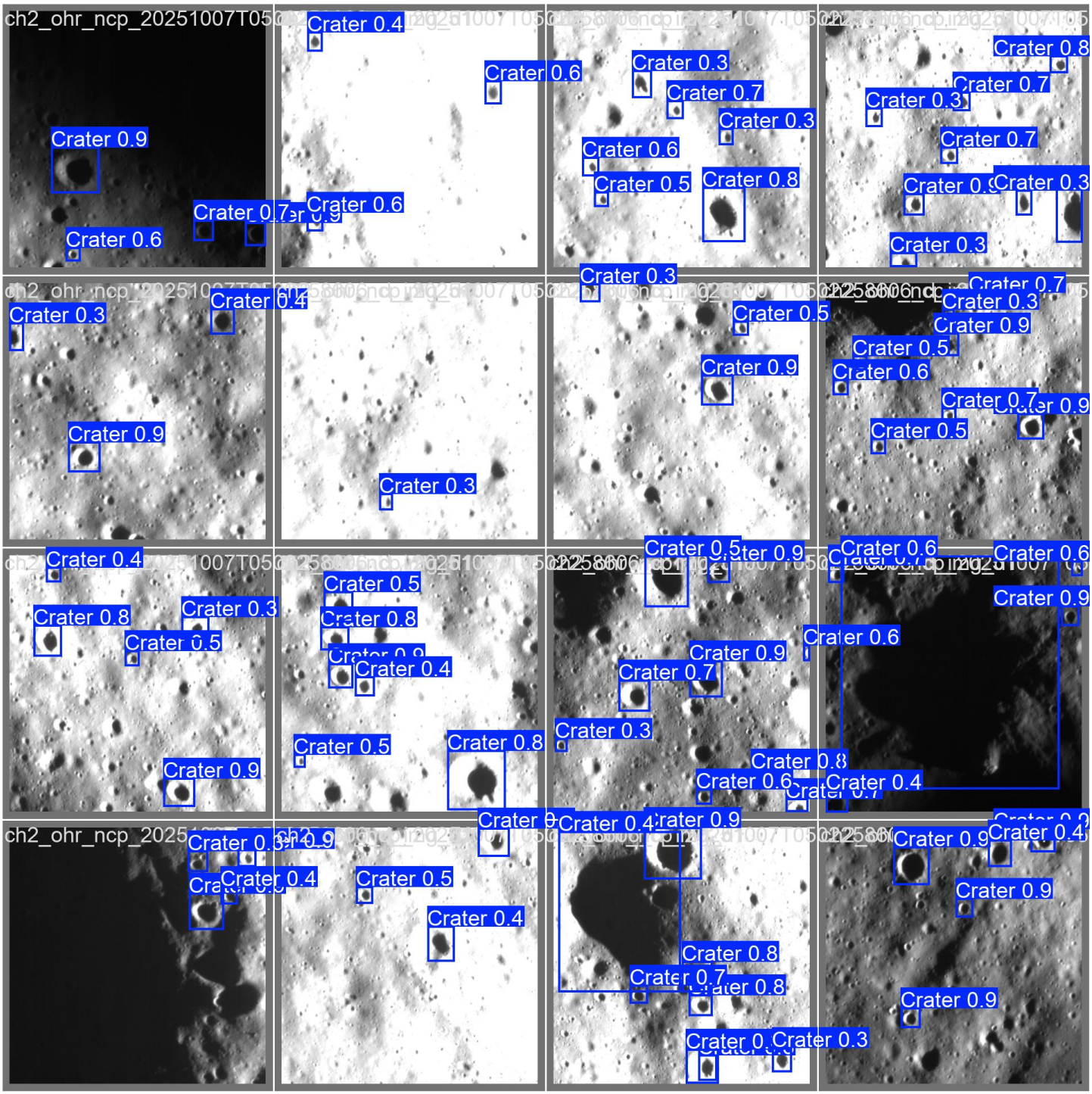
Lunar Crater Detection Pipeline & YOLO26 Analysis

Validation Batch 0: Ground Truth



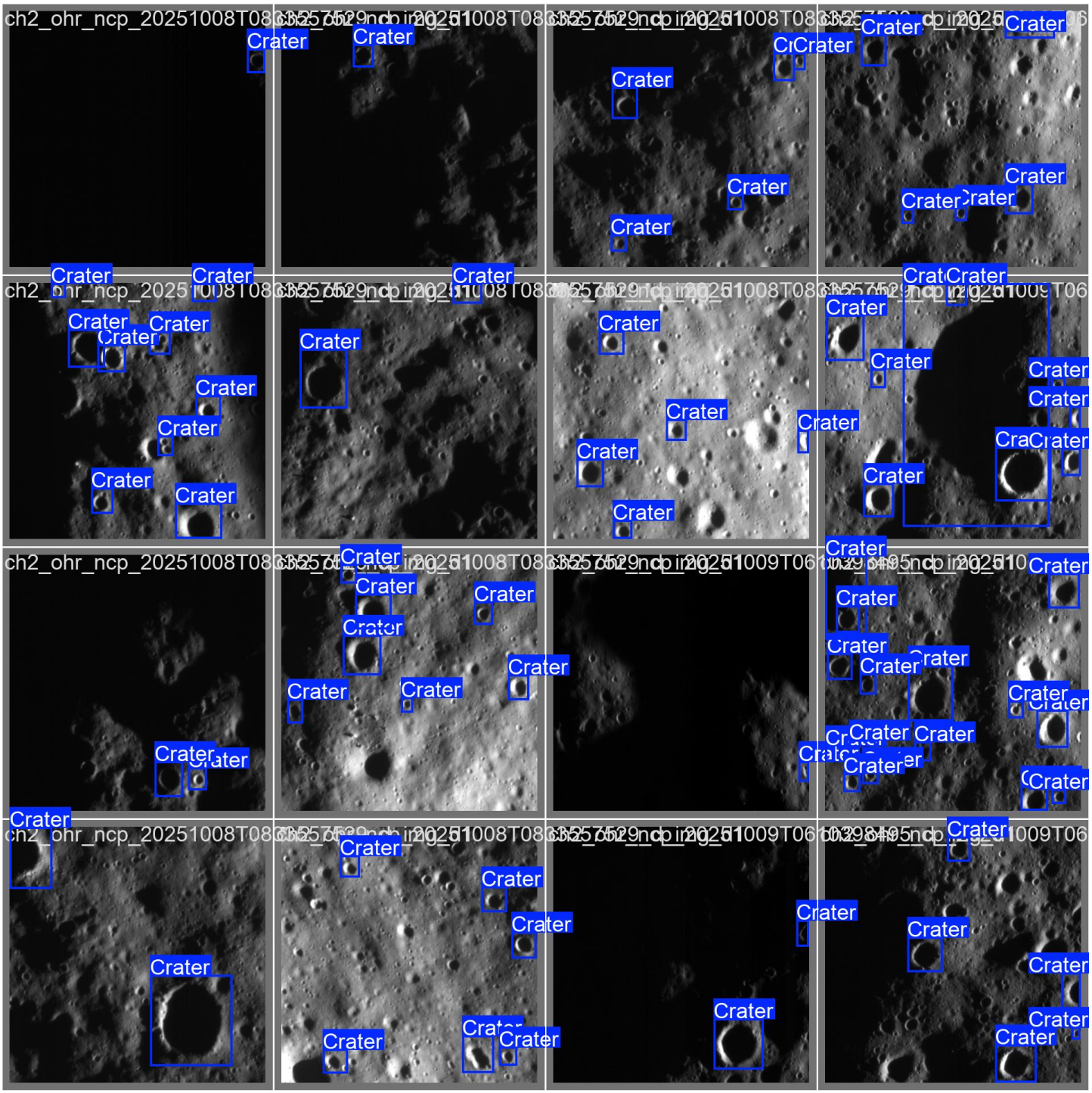
Lunar Crater Detection Pipeline & YOLO26 Analysis

Validation Batch 0: Predictions



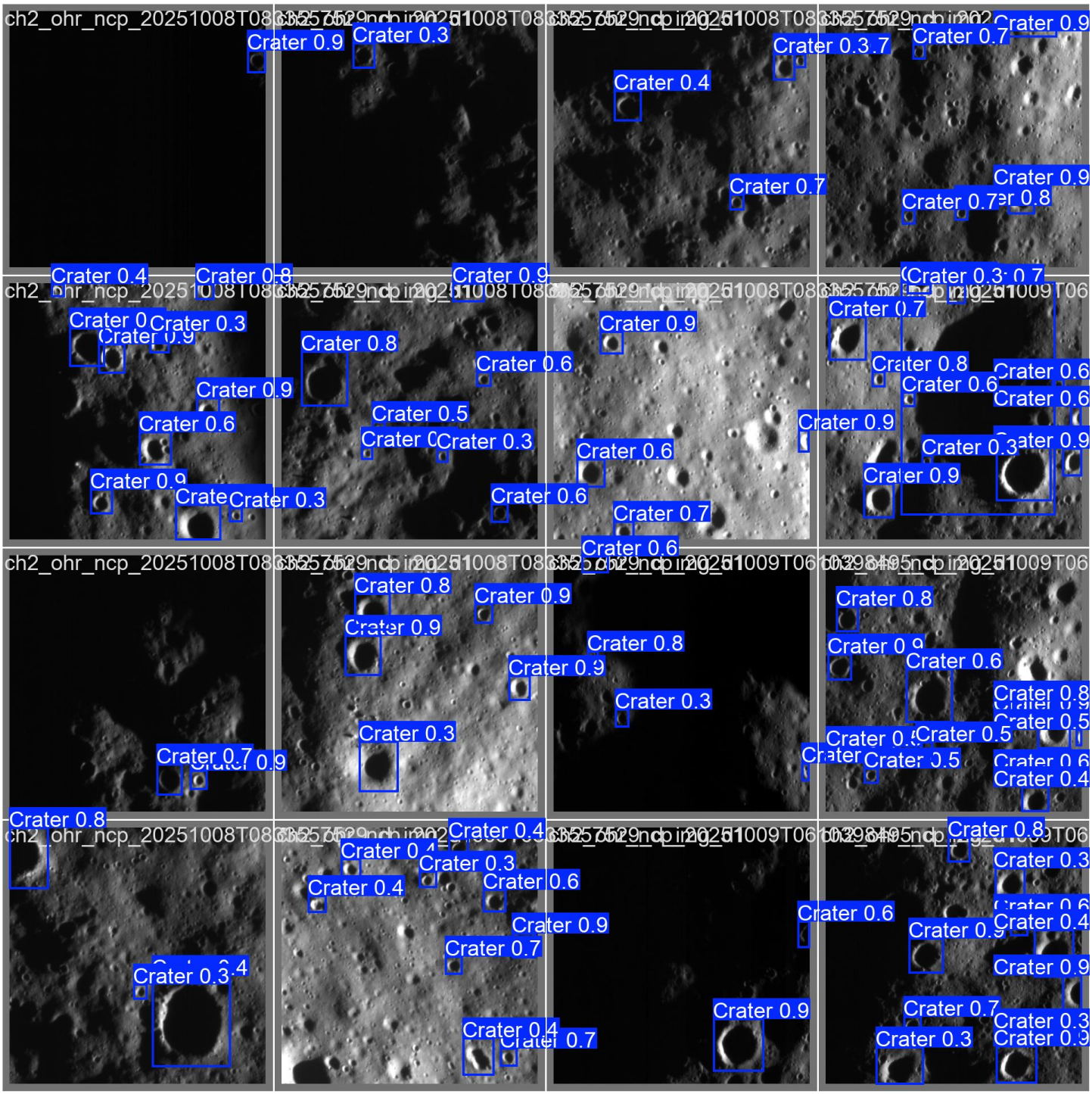
Lunar Crater Detection Pipeline & YOLO26 Analysis

Validation Batch 1: Ground Truth



Lunar Crater Detection Pipeline & YOLO26 Analysis

Validation Batch 1: Predictions



Lunar Crater Detection Pipeline & YOLO26 Analysis

Training Batch 0